NDP Final Ozeti

Yuksek Lisans - 14 Konu Tek Cikartmada

Nesneye Dayali Programlama
Final Sinavi Hazirlik Kitapcigi

2026

YUKSEK LISANS

İçindekiler

01. Temel Kavramlar: Sinif, Nesne ve NDP'nin 4 Ilkesi

02. C# Dil Temelleri (Sinavda Cok Cikar)

03. Sarmalama (Encapsulation) ve Erisim Denetleyiciler

04. Property (Ozellik) Kavrami

05. Sinif Uyeleri ve Kurucular (Constructor)

06. Reuse: Kodun Tekrar Kullanilabilirligi

07. Kalitim (Inheritance)

08. Interface, Abstract ve Static

09. Cok Bicimlilik (Polymorphism)

10. Tasarim Desenleri (Design Patterns)

1 Temel Kavramlar: Sinif, Nesne ve NDP'nin 4 Ilkesi

> Nesneye Dayali Programlama (NDP / OOP), gercek dunyadaki varliklari **nitelikleri** (alanlar) ve **davranislari** (metotlar) ile birlikte modelleyerek **siniflar** ve **nesneler** uzerinden kod yazma yaklasimidir.

1.1 Sinif vs Nesne

Sinif (Class)

Nesnelerin uretileceği **sablon**dur. Verileri (alan/property) ve bu verileri isleyecek davranislari (metot) birlikte barindirir. Sinif *soyut*tur; bellekte sadece bir nesne `new` ile uretildiginde yer kaplar.

Nesne (Object / Instance)

Bir siniftan `new` anahtar kelimesi ile uretilmis **somut** yapidir. Her nesnenin alan degerleri birbirinden bagimsizdir ve bellegin farkli yerlerinde tutulur.

Sinif ve Nesne Karsilastirmasi

Ozellik	Sinif	Nesne
Soyut/Somut	Soyut sablon	Somut ornek
Bellek	Tanimda yer kaplamaz	`new` ile heap'te yer alir
Sayisi	1 tane sinif tanimi	O siniftan istenildigi kadar nesne
Yapilis	`class Sinif { }`	`Sinif n = new Sinif();`

Pratikte siniflarin kendisi bellekte tek bir 'kod plani' olarak durur; degisen kisim nesneye ait alan degerleridir. Bu nedenle bir nesnedeki degisiklik diger nesneyi etkilemez. Sinavda bu fark genelde 'iki nesne olustur, birinin alanini degistir, ciktida ne olur?' seklinde olculur.

```C#
class Vasita
{
    public string marka;
    public string model;
    public int yil;
    public void BilgileriYazdir()
    {
        Console.WriteLine(yil + " " + marka + " " + model);
    }
}

// Kullanim
Vasita a1 = new Vasita();   // Nesne (instance) olusturma
a1.marka = "Opel";
a1.model = "Astra";
a1.yil = 2010;
a1.BilgileriYazdir();        // 2010 Opel Astra
```

Bu ornekte alanlar public birakildigi icin hizli anlatim yapiliyor; gercek uygulamada alanlar private tutulup property/metot ile disari acmak daha dogru yaklasimdir. Ozellikle dogrulama gerektiren alanlarda (yas, fiyat, kimlik no) public field ciddi bug riskidir.

> **PDF Alintisi**
>
> *Sinif soyut bir ifadedir ve nesneler olusuncaya kadar fiziksel olarak bellekte yer almazlar. Bir siniftan birden fazla nesne uretilebilir ve bu nesnelerin ozellikleri farkli olabilmektedir.*
>
> NDP_H01_02.pdf, Temel Kavramlar

1.2 NDP'nin 4 Temel Ilkesi

> **Neden Var?**
>
> Bir dilin **nesneye dayali** sayilabilmesi icin bu 4 ozelligi **tamamini** saglamasi gerekir. Eksik olan bir tanesi bile dili NDP sayfasinin disinda birakir.

NDP'nin 4 Temel Ilkesi

Ilke	Aciklama	C#'ta Karsiligi
Soyutlama (Abstraction)	Gereksiz detaylari dislayarak en oz haliyle modellemek; veriye erisim icin gerekli detaylari azaltma.	`abstract class`, `interface`, `property`
Sarmalama (Encapsulation)	Veri+kontrol soyutlamasinin birlikte yapilmasi; dis mudahaleleri kisitlama (data hiding).	Erisim denetleyiciler: `private`, `protected`, `public`, `internal`
Kalitim (Inheritance)	Bir sinifin baska sinifin uyelerini kendi uyesi gibi davranabilmesi; kod tekrar kullanimi.	C#: `:`, Java: `extends`
Cok Bicimlilik (Polymorphism)	Bir arayuz/sinifin kendisinden kalitim alan siniflari temsil edebilme yetenegi; kalitimin dogal sonucu.	`virtual` + `override`, `abstract` + `override`, base tipli parametre

Dort ilke birbirinden bagimsiz degildir: sarmalama soyutlamayi guclendirir, kalitim tekrar kullanim saglar, cok bicimlilik de kalitimdan gelen ortak arayuzu calisma aninda farkli davranislarla kullandirtir. Hocalar bu iliskiyi kurabildigini gorurse acik uclu soruda tam puan vermeye daha yakindir.

Sinavda Cikarsa

Bos doldurma: '... bir sinifin baska bir sinifin uyelerine erisebilmesidir' (Kalitim). Coktan secmeli: NDP'nin temel ilkelerinden hangisi degildir? (Compilation, Recursion gibi sahte secenekler). Acik uclu: '4 ilkeyi siralayip kisaca aciklayiniz'.

Elle Yazarken Tuyo

SSK-C mnemonigi: **S**oyutlama, **S**armalama, **K**alitim, **C**ok Bicimlilik. Sirala, her birine 1 cumle tanim, 1 ornek (orn. Sarmalama -> `private` alan + `public` property).

Yaygin Hata

Sarmalama'yi sadece `private` sanmak yanlistir; sarmalama veri+kontrol soyutlamasinin *birlikte* yapilmasidir. Sadece `private` ile *data hiding* olur; sarmalama icin erisim denetleyici + property/metot araciligi gerekir.

1.3 Yordamsal vs Nesneye Dayali

Iki paradigmanin temel farklari

Boyut	Yordamsal Programlama	Nesneye Dayali Programlama
Tasarim odagi	Fonksiyonlar (ne yapar?)	Nesneler (neler vardir?)
Veri organizasyonu	Ikincil	Birincil (sinifin icinde paketli)
Erisim kontrolu	Yok (global degiskenler)	Erisim denetleyicilerle korunur
Yeni tip uretme	Zor	`class` , `struct` , `interface`
Tekrar kullanim	Sinirli	`reuse` , kalitim, kompozisyon
Diller	Basic, Fortran, Pascal, C	C#, Java, C++

PDF Alintisi

Tasarim yapilirken sorulmasi gereken soru 'bu sistem ne is yapar?' degil, 'bu sistem hangi nesnelerden olusur?' olmalidir.

NDP_H06_Giris.pdf, s.3

2 C# Dil Temelleri (Sinavda Cok Cikar)

> Bos doldurma ve coktan secmeli sorularin onemli bir kismi C# temel sentaksindan gelir: value/reference tip, parse/convert, donguler, dizi sentaksi, operator sonuc tipi. Bunlar ezbere bilinmelidir.

2.1 Value vs Reference Tipler

Value vs Reference

Boyut	Value Tipler	Reference Tipler
Bellek bolgesi	**Stack**	**Heap** (stackte adres)
Ornekler	`bool, byte, sbyte, char, decimal, double, float, int, uint, long, ulong, short, ushort, struct, enum`	`class, interface, delegate, object, string`
Atama davranisi	Deger kopyalanir	Adres (referans) kopyalanir
Default deger	0/false/character 0	`null`

```C#
// Value tip: deger kopyalanir
int sayi3 = 6;
int sayi4 = sayi3;
sayi4 = 7;
Console.WriteLine(sayi3 + " " + sayi4);   // 6 7  (bagimsiz)

// Reference tip: ayni nesneye iki referans
class Sayi { public int Deger { get; set; } }
Sayi s1 = new Sayi();
s1.Deger = 4;
Sayi s2 = s1;            // Ayni nesne, iki ad
s2.Deger = 5;
Console.WriteLine(s1.Deger + " " + s2.Deger);  // 5 5
```

Buradaki kritik nokta: reference tipte degiskeni degistirmiyorsun, iki degiskenin gosterdigi tek bir nesnenin alanini degistiriyorsun. Value tipte ise her atamada yeni bir kopya vardir. Bu fark method parametrelerinde de ayni mantikla davranir.

Sinavda Cikarsa

Vize 2016 Soru-3.b'deki gibi: 'sayi1, sayi2' value tip degisikligi vs 'Sayi' class referans degisikligi. Ekran ciktisi sorusu olarak gelir.

2.2 Boxing ve Unboxing

Boxing

Value tipi (stack) heap alanina tasimak: `object x = i;` (i bir int ise).

Unboxing

Heap'teki object'i tekrar value tipe almak: `i = (int)x;` (cast gerektirir).

```C#
object x;
int i = 10;
x = i;              // BOXING (stack -> heap)

int j = (int)x;     // UNBOXING (heap -> stack), cast sart!
```

Yaygin Hata

Unboxing yaparken yanlis tipe cast denerseniz `InvalidCastException` firlatir.

2.3 Tip Donusumleri

Tip Donusum Yontemleri

Yontem	Kullanim	Hata Durumu
Implicit	Otomatik, kucukten buyuge (`int -> float`)	Yok
Explicit	`(int)f` seklinde cast	Veri kaybi olabilir
`int.Parse(s)`	String -> int	`FormatException`
`Convert.ToInt32(s)`	Cesitli kaynaklardan -> int	`FormatException` / `OverflowException`
`int.TryParse(s, out i)`	Guvenli, bool doner	Hata firlamaz, false doner

Sinav acisindan guvenli tercih: kullanicidan gelen veri icin Parse yerine TryParse. Cunku TryParse hata firlatmadan akisi devam ettirir. Parse/Convert daha kisa ama hatali giriste programi durdurabilir; bu da event tabanli sorularda puan kaybina yol acar.

2.4 Aritmetik Sonuc Tipi (Tipik tuzak!)

```C#
int sayi1 = 5, sayi2 = 2;
float sonuc = sayi1 / sayi2;        // sonuc = 2.0f  (kalan atilir!)

float sonuc2 = (float)sayi1 / sayi2; // sonuc2 = 2.5f
```

Yaygin Hata

Iki `int` 'in bolumu **int**'tir. Sonucu `float` 'a atasaniz da kalan atilmis halde ata - 2.5 degil 2 olur. Cozum: en az birini cast et.

2.5 Donguler ve Karar Yapilari Kisa Hatirlatici

```
C#
// Standart donguler
for (var i = 0; i < 5; i++) { ... }
while (kosul) { ... }
do { ... } while (kosul);

foreach (var x in koleksiyon) { ... }    // koleksiyon IEnumerable olmali

// break vs continue
for (var i = 0; i < 5; i++) {
    if (i == 2) break;     // dongusu komple terk eder
    if (i == 2) continue;  // sadece bu turu atlar
}

// switch (her case sonunda break / return sart)
switch (gun) {
    case 1: ad = "pazartesi"; break;
    default: ad = "HATA"; break;
}
```

Dongu sorularinda hoca genelde break/continue veya dongu kosulu ile oynayip ciktinin kac olacagini sorar. Hizli cozum teknigi: her iterasyonda sadece degisen degiskenleri satir satir not et, kagitta mini tablo cikar. Bu teknik uzun kodlarda hatayi ciddi azaltir.

2.6 Diziler

```
C#
// Tek boyutlu
int[] dizi1 = new int[10];
int[] dizi2 = { 1, 2, 3 };

// Cok boyutlu (rectangular)
int[,] matris = { {1, 2}, {3, 4}, {10, 11} };

// Duzensiz (jagged) - dizilerden olusan dizi
int[][] jagged = new int[3][];
jagged[0] = new int[4] { 0, 1, 2, 3 };
jagged[1] = new int[2] { 4, 5 };
```

> **Elle Yazarken Tuyo**
>
> Kafa karistirici: `int[,]` dortgen matris (her satir esit), `int[][]` ise duzensiz (her satirin uzunlugu farkli olabilir). Sinavda dikkat!

3 Sarmalama (Encapsulation) ve Erisim Denetleyiciler

> Sarmalama, bir nesne uzerinde **veri** ve **kontrol** soyutlamasinin birlikte yapilmasidir. Amac: nesneyi dis mudahalelerden korumak, kullanim amacina uygun davranmasini saglamak.

3.1 Erisim Denetleyiciler

C# Erisim Denetleyiciler

Anahtar Kelime	Erisim Alani
`public`	Her yerden erisilebilir.
`private`	Sadece tanimlandigi sinif icinden.
`protected`	Tanimlandigi sinif + ondan kalitim alan siniflar.
`internal`	Ayni **assembly** icindeki tum siniflar (C#).
`protected internal`	Hem protected hem internal (ikisinin *birlesim* kumesi).

Akilda Kalsin

5 husus (bos doldurmada gelebilir): (1) Isletim sistemi public/private bakmaz - hafiza takibi ile her sey okunur. (2) Guvenligi programci saglar. (3) C#'in default'u `private`, Java'nin `public`. (4) Alanlari (`field`) `public` yapmak *onerilmez*. (5) Sinifin kendisini `private` yapmak *derleme hatasidir*.

3.2 Veri Saklama (Data Hiding) Ornegi

```C#
// KOTU: alani public yapmak NDP'ye aykiri
public class Ornek
{
    public int yas;    // istenmeyen erisim acik
}
// Kullanici: o.yas = -30; (anlamsiz ama derlenir)

// IYI: alani private + property ile sarmalanmis
public class Ornek
{
    private int yas;          // veri soyutlamasi
    public int Yas             // kontrol soyutlamasi
    {
        get { return yas; }
        set
        {
            if (value < 0) yas = 0;
            else yas = value;
        }
    }
}
```

Data hiding sadece 'gizlemek' degil, sistemi yanlis kullanimdan korumaktir. Disaridan dogrudan alan atamasi yerine property ile tek bir giris noktasi verince hem validasyon hem loglama hem de gelecekteki kural degisiklikleri tek yerde yonetilir.

PDF Alintisi

Bir sinif icinde bulunan uyelerin, o sinifi kullananlar tarafindan erisilip erisilemeyecegini belirlemeye yarar.

NDP_H04_01.pdf, Sarmalama

Sinavda Cikarsa

Bos doldurma: 'Sinif uyelerine erisim duzeyini belirleyen anahtar kelimelere ... denir.' (erisim denetleyicileri). MCQ: hangisi C#'ta gecerli degildir? (orn. `protected internal` 'i tani; `friend` C# kelimesi degildir).

Elle Yazarken Tuyo

Elle kod yazdirilirsa, bir sinif tasarimi sorulursa **once alan, sonra property, sonra kurucu, sonra metot** sirasini koru. Alanlari mutlaka `private` yap (otomatik puan), property ile dis dunyaya ac.

4 Property (Ozellik) Kavrami

> Property, alanlarin `public` aciklanmasinin sakincalarini ortadan kaldirir. Veri kabul edilmeden once **kontrol** imkani verir; sadece okunur veya sadece yazilir hale getirilebilir.

4.1 Tam Property

```
C#
public class Ornek
{
    private int yas;          // backing field

    public int Yas             // property
    {
        get { return yas; }    // okuma
        set                     // yazma (value = atanan deger)
        {
            if (value < 0)
                yas = 0;
            else
                yas = value;
        }
    }
}
```

Tam property ile auto-property arasindaki secim kriteri basit: dogrulama, formatlama, yan etki veya hesaplama varsa tam property; sadece veri tasima varsa auto-property. Sinavda validation kosulu goruyorsan auto-property yerine tam property yazmalisin.

4.2 Otomatik Property (C# 3.0+)

```
C#
public class Yeni
{
    public string Adi { get; set; }   // backing field derleyici tarafindan uretilir
    public int Id    { get; private set; }   // disaridan sadece okunur
}
```

4.3 Sadece Okunabilir / Sadece Yazilabilir

C#

```
// Sadece okunur (read-only)
public class Daire
{
    private int yaricap = 20;
    public int Yaricap
    {
        get { return yaricap; }
        // set yok!
    }
}
// d.Yaricap = 100;  // DERLEME HATASI
```

C#

```
// Sadece yazilir (write-only) - nadir
public class Setter
{
    private int m_A;
    public int A
    {
        set { m_A = value; }
        // get yok!
    }
}
// int x = s.A;  // DERLEME HATASI
```

4.4 Property ile Veri Dogrulama (KLASIK SINAV SORUSU)

```C#
// Vize 2016 - "Uretici" property ornegi (switch-case ile)
public string Uretici
{
    get { return marka; }
    set
    {
        switch (value)
        {
            case "Toyota":
            case "Opel":
                marka = value;
                break;
            default:
                marka = "Diger";
                break;
        }
    }
}

// Final 2018-2019 - DemID property ornegi (7 karakter, ilk 2 "DE")
private string demID;
public string DemID
{
    get { return demID; }
    set
    {
        if (value.Length == 7 && value.StartsWith("DE")
            && value.Substring(2).All(char.IsDigit))
            demID = value;
        else
            demID = "";
    }
}
```

Sinavda Cikarsa

Bos doldurma: 'Property'nin set blogunun parametre ismi otomatik olarak `...` 'dur.' (value). Elle kod: 'X property'sini su kosullarla yaziniz' (tipik 10-15 puan).

Elle Yazarken Tuyo

Property sirasi: (1) **backing field** (private alan), (2) **property bloku** (ayni veri tipi, baş harfi buyuk isim), (3) icine `get` ve `set`. `set` icinde `value` her zaman atanan degeri tutar. **Validation** sorulursa: set icinde `if` ile kontrol, gecerse ata, gecmezse default deger ver.

Yaygin Hata

`set` icinde yanlislikla `this.X = value;` yazmak: `StackOverflowException` (sonsuz recursion). Dogrusu `field`'a (kucuk harfli backing) atamaktir.

5 Sinif Uyeleri ve Kurucular (Constructor)

> Sinifin tum uyelerini siralayalim: **Alan (field), Property, Metot, Olay (event), Temsilci (delegate), Kurucu (constructor), Yikici (destructor), Indexer**. Hepsinin sinav formulasyonu bilinmelidir.

5.1 Sinifin Uyeleri (Members)

C# Sinif Uyeleri (Event/Delegate bu final kapsaminda sorulmaz)

Uye	Aciklama	Sentaks ozeti
Field	Sinifin saklayacagi veri.	`private int yas;`
Property	Disaridan controlle erisim.	`public int Yas { get; set; }`
Metot	Davranis / fonksiyon.	`public int Topla(int a, int b) { ... }`
Kurucu	Nesne olusurken otomatik calisir.	`public Sinif() { ... }`
Yikici	Nesne yok edilirken (C# nadir).	`~Sinif() { ... }`
Indexer	Sinifi dizi gibi kullanma.	`public T this[int i] { ... }`
Event *(sinav disi)*	Olay yayinlama.	`public event EventHandler X;`
Delegate *(sinav disi)*	Metot tutucu tip.	`delegate void T();`

5.2 Kurucular (Constructor)

> **Kurucu**
>
> Sinifla **ayni isimde**, geri donus tipi olmayan (gercekte sinifin kendisini doner), `new` ile bir nesne olustugu anda **otomatik calisan** ozel metottur.

Akilda Kalsin

Kurucularin Ozellikleri:

- Sinifla ayni isimdedir.
- Geri donus degeri yoktur (yazilmaz).
- **Overload** edilebilir (parametre farkliligi ile birden cok kurucu).
- Belirtilmese bile her sinifin **varsayilan (parametresiz) kurucusu** vardir.
- Erisim belirteci *alir* (genelde `public`).

```C#
public class Yamuk
{
    private double taban, tavan, yukseklik;

    // Parametresiz (varsayilan)
    public Yamuk()
    {
        this.taban = 6; this.tavan = 4; this.yukseklik = 7;
    }

    // Parametreli (overload)
    public Yamuk(double taban, double tavan, double yukseklik)
    {
        this.taban = taban;
        this.tavan = tavan;
        this.yukseklik = yukseklik;
    }
}

// Kullanim
Yamuk y1 = new Yamuk();              // parametresiz
Yamuk y2 = new Yamuk(8, 3, 6);       // parametreli
```

Kurucu sorularinda en cok unutulan nokta, property ile atanmasi istenen alanlarin dogrudan field'a degil property'ye atanmasidir. Boylece kurucu icinde de validation devreye girer. Ozellikle marka, ID, tarih gibi kisitli verilerde bu detay puan getirir.

5.3 this Anahtar Kelimesi

- Varsayilan sinifin **kendi nesnesini** referans eder.
- Ayni isimde parametre ile alan ayirmak icin (`this.taban = taban;`).
- Baska metoda nesneyi parametre olarak gondermek icin.
- Sinifi **indexer** olarak kullanmak icin (`public T this[int i]`).

5.4 Static Kurucular

Akilda Kalsin

Static kurucular sinifin **ilk dinamik nesnesi olusmadan once**, sinifa ilk erisildiginde **bir defa** calisir. Ozellikleri:

1. Bir sinifin **en fazla 1** static kurucusu olabilir.
2. Erisim belirteci **yazilmaz**.
3. Parametre **almaz**.
4. Sadece static uyelere erisebilir.
5. Hem static hem normal kurucu varsa once static calisir.

```C#
class StatikSinif
{
    static DateTime zaman = DateTime.Now;

    static StatikSinif()              // Static kurucu
    {
        Console.WriteLine(zaman);
    }

    public StatikSinif()              // Normal kurucu
    {
        Console.WriteLine("Sonra calisir");
    }
}
```

5.5 Yikicilar (Destructor)

- `~SinifAdi()` seklinde yazilir, **parametre almaz**.
- Bir sinifin **en fazla 1** yikicisi olabilir.
- Overload **edilemez**.
- Kalitim yolu ile **gecmez**.
- Manuel cagirilmaz, GC tarafindan otomatik calisir.
- Sadece **class**'larda gecerlidir, `struct`'larda yoktur.

5.6 Metot Asiri Yukleme (Overload)

Overload

Ayni isimli, fakat **parametre tipi/sayisi farkli** metotlardir. Sadece geri donus tipinin degisimi overload *yapmaz*.

```C#
class Toplama
{
    public int Sonuc(int a) { return a + a; }
    public double Sonuc(double a, double b) { return a + b; }
    public string Sonuc(string a) { return a; }
    // HATA: sadece geri donus tipinin farkli olmasi yetmez
    // public double Sonuc(int a) { ... }  // CAKISIR
}
```

Sinavda Cikarsa

Bos doldurma: 'Ayni isimli, farkli parametreli metotlara ... denir.' (asiri yukleme/overload). MCQ: Overload kuralinda gecerli olmayan: 'Sadece donus tipi farkli olabilir'. (yanlistir)

Elle Yazarken Tuyo

Kurucu sirasi (elle kod):

1. Sinif imzasi: `class X`
2. `{`
3. Alanlar (private)
4. Property'ler
5. Kurucu(lar): parametresiz, sonra parametreli
6. Diger metotlar
7. `}`

Her kurucu icinde once `this.X = X;` seklinde atama yap. Sinav puanini bu siralama kurtarir.

6 Reuse: Kodun Tekrar Kullanilabilirligi

> NDP'nin temel hedefi **reuse**. Bir kodu defalarca yazmak yerine: ortak siniflar, `base` sinif kalitimi, metot cikartma, kompozisyon. *'Her sinifin tek gorevi olmalidir'* (Single Responsibility Principle).

6.1 Ilkeler

- **Sorumluluk ayrimi**: Yamuk'un alanini hesaplamak `Program.cs`'in degil `Yamuk.cs`'in isidir.
- **Degisime kapali, gelisime acik**: Yeni sekil eklerken eski kodlara dokunulmaz.
- **Encapsulation**: `AlanHesapla()` sadece sinif icinden cagriliyorsa `private` yap.
- **Kalitim+virtual**: ortak alan/davranisi base sinifa cek, ozel davranislari override et.

```C#
// Reuse'un dogal sonucu: SekilBase + alt siniflar
class SekilBase
{
    protected double r, h;
    public SekilBase() { }
    public SekilBase(double r, double h) { this.r = r; this.h = h; }
    public virtual double AlanHesapla() { return 0; }
    public void EkranaYazdir()
    {
        Console.WriteLine(this.GetType().Name + " Alani = {0:F2}", AlanHesapla());
    }
}
class Daire : SekilBase
{
    public Daire(double r) : base(r, 0) { }
    public override double AlanHesapla() { return Math.PI * r * r; }
}
class Dikdortgen : SekilBase
{
    public Dikdortgen(double en, double boy) : base(en, boy) { }
    public override double AlanHesapla() { return r * h; }
}
// Yeni sekil eklemek = sadece yeni sinif eklemek (eski koda dokunma!)
```

Bu yapi ayni zamanda Open/Closed Principle'a ornek olur: mevcut siniflari degistirmeden yeni sekil eklenebilir. Kod buyudukce bu prensip bakim maliyetini dusurur. Soru metninde 'sisteme yeni sinif ekleyince eski kod degismesin' geciyorsa bu yaklasimi yaz.

Sinavda Cikarsa

Acik uclu: 'Verilen koddaki sorumluluk dagilimini iyilestirin.' veya 'NDP'nin tekrar kullanilabilirligi nasil saglar?' Cevap: kalitim, kompozisyon, polimorfizm, abstract base.

7 Kalitim (Inheritance)

> Kalitim, bir sinifin baska bir sinifin uyelerini kendi uyesi gibi davranabilmesidir. Amac: ortak kodu tek bir yerde tutmak (reuse) ve merkezi yonetim saglamak.

7.1 Temel Sentaks

Kalitim Sentaksi

Dil	Sentaks	Cokli kalitim
C#	`class B : A { }`	Tek sinif + cok interface
Java	`class B extends A { }`	Tek sinif + cok interface
C++	Iki nokta uste	Coklu sinif kalitimi **destekler**

```
C#
class BaseClass
{
    public string adi;
    protected string soyAdi;
    private string meslek;            // turemiste GORUNMEZ
    public void Yaz() { Console.WriteLine("Base"); }
}

class DerivedClass : BaseClass
{
    public void Goster()
    {
        adi = "x";        // PUBLIC: erisim var
        soyAdi = "y";     // PROTECTED: erisim var
        // meslek = "z";  // PRIVATE: HATA, erisim yok
    }
}
```

7.2 base Anahtar Kelimesi

- Turemis siniftan **temel sinifin** uyelerine erismek icin: `base.X`.
- Turemis sinifin kurucusu, temel sinifin kurucusunu cagirmak icin: `: base(args)`.
- Temel sinifta override edilmis bir metot icinde `base.Yaz()` ile asili davranisi cagirma.

```C#
class A
{
    public A(int a, int b)
    {
        Console.WriteLine(a + b);
    }
}

class B : A
{
    // B'nin kurucusu A'nin kurucusuna deger gondermek zorunda
    public B(int a) : base(a, 10)
    {
        Console.WriteLine(a);
    }
}

// new B(5) cagirildiginda:
//   1) once A(5, 10) calisir -> 15
//   2) sonra B(5) govdesi -> 5
```

7.3 virtual / override / new / sealed

Kalitim Anahtar Kelimeleri

Anahtar	Yeri	Davranis
`virtual`	Temel sinifta metot/property	Turemis sinifa override izni verir.
`override`	Turemis sinifta metot/property	Temel sinifin virtual/abstract uyesini ezer (gec baglama).
`new`	Turemis sinifta metot	Temel sinifin metodunu *gizler* (erken baglama, polimorfizm yok).
`sealed`	Sinif veya override edilmis metot	Daha fazla kalitim/override engellenir.
`abstract`	Sinif veya metot	Govdesiz; turemiş sinif override etmek zorunda.

```C#
class GenelSinif
{
    public virtual void EkranaYazdir()
    {
        Console.WriteLine("Base");
    }
}

class TuremisSinif : GenelSinif
{
    public override void EkranaYazdir()        // virtual + override = polymorphism
    {
        Console.WriteLine("Derived");
    }
}

// new ile gizleme (override DEGIL)
class A { public void X() { Console.WriteLine("A"); } }
class B : A { public new void X() { Console.WriteLine("B"); } }
// A a = new B(); a.X();  -> "A" yazar (gec baglama yok)
```

Virtual/override ile new farkini cikti sorusunda ayirmak icin referans tipine bak: override varsa gercek nesnenin metodu, new ile gizleme varsa referans tipinin metodu calisir. Bu ayrim Final sorularinda en cok puan kaybettiren noktalardan biridir.

7.4 Sealed ve Static Sinif

Sinif Turleri

Tip	Kalitim verir	Nesne uretilir
`sealed class`	**HAYIR**	Evet
`static class`	**HAYIR**	**HAYIR**
`abstract class`	EVET (zorunlu)	**HAYIR**

Sinavda Cikarsa

Klasik bos doldurma: 'Bir sinifin baska sinifa kalitim vermesini engellemek icin ... anahtar kelimesi kullanilir.' (sealed). MCQ: 'sealed siniflar icin hangisi yanlistir?'.

Elle Yazarken Tuyo

Override yapmak icin SART: Temel sinifta `virtual` ya da `abstract` olmali. Aksi durumda `new` kullanman gerekir (uyari). Sinavda 'kalitim ver, sonra X metodunu override et' denirse, base'de `virtual` yazmayi sakin unutma.

Yaygin Hata

`abstract` sinifdan nesne uretilemez (Final 2016 Soru-5'te tam bu sorulmus). `sealed` sinif kalitim alinamaz (ayni sorunun 7. veya 16. satiri).

8 Interface, Abstract ve Static

> Final sinavinda **en cok karsilastirma sorusu** burada gelir. Abstract vs Interface ezbere bilinmesi gereken bir farktir.

8.1 Interface (Arayuz)

Interface

Siniflar arasi **kontrat** niteliginde, govdesiz metot/property/event/indexer deklarasyonlarindan olusan yapidir. Adi gelenek olarak **I** ile baslar (orn. `IComparable`).

Akilda Kalsin

Interface kurallari:

- Govdesi **bos** uyeler tutar (metot/property/event/indexer).
- Tum uyeleri otomatik `public`'tir; *yazilmaz*.
- **Alan (field)** bulunduramaz.
- **Kurucu / yikici** bulunduramaz.
- **Static** uye bulunduramaz.
- `new` ile nesne uretilemez.
- Bir interface baska interface'lerden kalitim alabilir.
- Bir sinif **birden cok interface**'den kalitim alabilir (C#).
- Interface bir sinifdan **kalitim alamaz**.

```C#
interface IOrnek
{
    // Property (govdesiz)
    double X { get; set; }
    double Y { get; set; }

    // Metot (govdesiz)
    double Hesapla();

    // Sadece okunabilir property
    string Ad { get; }

    // Indexer (sadece C#)
    string this[int index] { get; set; }
}

class Deneme : IOrnek
{
    public double X { get; set; }     // tum uyeleri yazmak zorunda
    public double Y { get; set; }
    public string Ad => "Ornek";
    public string this[int index] { get; set; }
    public double Hesapla() { return X * Y; }
}
```

Interface implementasyonunda imzalar birebir eslesmelidir: donus tipi, parametre sayisi ve tipleri ayni olmali. Erisim belirteci de implement eden sinifta public olmali; aksi halde 'daha kisitli erisim' derleme hatasi alirsin.

8.2 Cokli Interface Implementasyonu (Final 2018-2019 ilgili)

```C#
interface IMusteri { void EkranaYazdir(); }
interface ISiparis { void EkranaYazdir(); }   // ayni isim!

class Fatura : IMusteri, ISiparis
{
    // Ayni isimli iki metot iste: explicit implementation kullan
    void IMusteri.EkranaYazdir()
    {
        Console.WriteLine("Musteri bilgisi");
    }
    void ISiparis.EkranaYazdir()
    {
        Console.WriteLine("Siparis bilgisi");
    }
    // Cagri: ((IMusteri)f).EkranaYazdir();
}
```

8.3 Abstract Sinif

Abstract Sinif

`new` ile nesnesi uretilemeyen, sadece **kalitim verilmek** uzere tanimlanan siniftir. Hem govdeli hem govdesiz (abstract) uyeler barindirabilir.

- `abstract class X` seklinde tanimlanir.
- Icinde **abstract metot/property** olabilir (govdesiz, override *zorunlu*).
- Abstract uye yazabilmek icin sinifin abstract olmasi **sart**tir.
- Abstract uyeler `public` veya `protected` olmali (`private` olamaz).
- Kalitim alan sinif tum abstract uyeleri `override` ile yazmak zorunda.
- Static sinif ve interface'den kalitim **almaz**.

C#

```
abstract class SekilBase
{
    protected double R;
    protected double H;

    public SekilBase() { }
    public SekilBase(double r, double h) { R = r; H = h; }

    protected abstract double AlanHesapla();    // govdesiz, override SART

    public void EkranaYazdir()
    {
        Console.WriteLine(GetType().Name + " Alani = " + AlanHesapla());
    }
}

class Daire : SekilBase
{
    public Daire(double r) : base(r, 0) { }
    protected override double AlanHesapla()   // override SART
    {
        return Math.PI * R * R;
    }
}
```

8.4 ABSTRACT vs INTERFACE - Klasik Karsilastirma

Abstract vs Interface

Boyut	Abstract Sinif	Interface
Govdeli uye	Olabilir	**Olamaz** (default impl. C# 8 hariç)
Alan (field)	Olabilir	**Olamaz**
Kurucu	Olabilir	**Olamaz**
Erisim belirteci	Yazilir	**Yazilmaz** (otomatik public)
Static uye	Olabilir	**Olamaz**
Cokli kalitim	C#'ta **HAYIR**	EVET (cok interface)
Nesne uretimi	**HAYIR**	**HAYIR**
Isimlendirme	Genelde 'Base' eki	**I** ile baslar (IComparable)
Kullanim	Ortak govde+sozlesme	Sadece sozlesme

Kisa karar kurali: 'ortak kod + zorunlu iskelet' gerekiyorsa abstract class; 'sadece sozlesme, farkli hiyerarsilerde birlikte calisma' gerekiyorsa interface. Modern C#'ta default interface method olsa da ders baglaminda interface'i govdesiz dusunmek daha guvenli.

8.5 Static

- Static uyelere sinifin nesnesini olusturmadan erisilir: `SinifAdi.Uye`.
- Static sinif: nesnesi olusturulamaz, kurucu/yikici alamaz, kalitim almaz/vermez.
- Static metot icinden **sadece** static uyelere erisilebilir.
- Static olmayan metot icinden static uyeye erisilebilir.
- Static kurucu sinifa ilk erisildiginde 1 defa calisir.

```C#
static class TemelAritmetik
{
    public static double Topla(double a, double b) { return a + b; }
    // public string adi;       // HATA: static olmayan uye olamaz
    // public TemelAritmetik() { } // HATA: kurucu olamaz
}

// Kullanim
double s = TemelAritmetik.Topla(1, 2);
// TemelAritmetik t = new TemelAritmetik();  // HATA
```

Sinavda Cikarsa

Final 2018-2019'da 30 sorunun cogu bu uc kavramdan: (11) sealed, (12) partial, (13) virtual asla metot adi onunde olmaz, (14) interface birden cok sinifdan kalitim alamaz, (15) I ile baslar, (16) kurucu/yikici olamaz, (17) interface erisim belirteci almaz, (18) abstract nesne olusturulamaz, (19) hem abstract hem normal uye olabilir, (20) abstract metot sadece abstract sinifta, (21) override SART, (22) genelde A ile baslar - tartismali, (23) static sinif kalitim almaz, (24) ayni sinifin static-nonstatic kurallari, (25) Sinifadi.Metot, (26) kurucu erisim belirteci alir mi (alir!), (27) hicbiri (kalitimda kurucu mecbur degildir).

Elle Yazarken Tuyo

ABSTRACT vs INTERFACE ezber kartelasi:
Abstract: *govde olabilir*, alan olabilir, kurucu olabilir, tek kalitim
Interface: *hep bos*, alan yok, kurucu yok, cok kalitim, isim I'yla baslar

9 Cok Bicimlilik (Polymorphism)

> Bir arayuzun ya da sinifin **kendisinden kalitim alan siniflari temsil edebilme yetenegidir**. Kalitimin dogal sonucudur. Ancak **kalitim + override + gec baglama** birlikte calismadan polimorfizm olmaz.

9.1 Erken vs Gec Baglama

Erken vs Gec Baglama

Boyut	Erken Baglama (Early)	Gec Baglama (Late)
Karar zamani	Derleme aninda	Calisma aninda
Anahtar	`new`, dogrudan cagri	`virtual` + `override`
Polymorphism	Yok	**Var**
Hangi metot calisir?	*Referans* tipinin metodu	*Nesne*nin gercek tipinin metodu

```C#
abstract class HayvanBase
{
    public string Adi { get; }
    public HayvanBase(string adi) { Adi = adi; }
    public abstract string SesVer();
}

class Kedi : HayvanBase
{
    public Kedi(string adi) : base(adi) { }
    public override string SesVer() { return "Miyaw"; }
}

class Kopek : HayvanBase
{
    public Kopek(string adi) : base(adi) { }
    public override string SesVer() { return "Hav"; }
}

// POLIMORFIZM: HayvanBase referansi, gercek tipe gore davranis
HayvanBase[] bakimEvi = { new Kopek("Karabas"), new Kedi("Pamuk") };
foreach (HayvanBase h in bakimEvi)
    Console.WriteLine(h.SesVer());   // Hav, Miyaw
```

Polimorfizmin gucu burada gorunur: cagri noktasi degismeden (h.SesVer), davranis nesnenin gercek tipine gore degisir. Yeni bir sinif (Kus) eklendiginde foreach blogunu degistirmeden sistem calismaya devam eder.

9.2 Upcast / Downcast

```C#
// Upcast (turemis -> temel): IMPLICIT (gizli), guvenli
HayvanBase hb = new Kedi("Pamuk");

// Downcast (temel -> turemis): EXPLICIT cast gerekir, riskli
Kedi k = (Kedi)hb;                    // hb gercekten Kedi ise OK
// Kopek kp = (Kopek)hb;              // InvalidCastException calisma aninda

// Guvenli: 'is' veya 'as' operatoru
if (hb is Kedi kedi) { kedi.Miyavla(); }    // pattern matching (C# 7+)
Kopek? maybeKopek = hb as Kopek;            // basarisizsa null
```

9.3 Polimorfizm Kullanim Sebepleri

- Yeni alt sinif eklemek kolay (eski kod degismez).
- Genel metotlar tek bir base/interface uzerinden hepsiyle calisir.
- Koleksiyonlari heterojen tutabilirsin (`List<HayvanBase>`).
- Strategy/Template Method gibi tasarim desenlerinin temelidir.

Sinavda Cikarsa

Final 2018-2019 Soru-28: 'Cok bicimlilik icin asagidakilerden hangisi soylenebilir?' Cevap: 'Bir sinifin baska sinifin yerini alabilmesidir' + 'Kalitimin dogal sonucudur'.

Elle Yazarken Tuyo

Eger sorulda 'gec baglama / late binding' geciyorsa anahtar kelimeler `virtual` + `override` olmali. Sadece `new` ile gizlerseniz polymorphism calismaz (sorun bu olur).

9.4 Sinavda SIK Cikan Ek Konular (Eksik Kalmasin)

> Asagidaki dort baslik, bosluk doldurma + test + elle kod sorularinda cok tekrar eder: **Exception handling**, **Indexer**, **Operator Overload**, **is/as ile guvenli cast**.

9.4.1 Exception Handling - Catch Sirasi ve Dogru Kalip

```
C#
try
{
    int sonuc = sayi1 / sayi2;
    Console.WriteLine(sonuc);
}
catch (DivideByZeroException)
{
    Console.WriteLine("Sifira bolme hatasi");
}
catch (FormatException)
{
    Console.WriteLine("Sayisal format hatasi");
}
catch (Exception)
{
    Console.WriteLine("Genel hata");
}
finally
{
    Console.WriteLine("Kaynak kapatma / son isler");
}
```

Akilda Kalsin

Kural: catch bloklari *ozelden genele* yazilir. Genel `Exception` once yazilirsa alttaki ozel catch'ler ulasilamaz olur.

9.4.2 Indexer - Sinifi Dizi Gibi Kullanma

```
C#
class NotDefteri
{
    private int[] notlar = new int[5];

    public int this[int i]
    {
        get { return notlar[i]; }
        set { notlar[i] = (value < 0 || value > 100) ? 0 : value; }
    }
}

var n = new NotDefteri();
n[0] = 120;                // validation -> 0
Console.WriteLine(n[0]);   // 0
```

Elle Yazarken Tuyo

Indexer imzasinda metot adi yoktur; mutlaka `this[...]` kullanilir. Bu sentaks sinavda ayirt edici puandir.

9.4.3 Operator Overload - Mutlaka static

```
C#
class Veri
{
    public int Deger { get; set; }
    public Veri(int d) { Deger = d; }

    public static Veri operator +(Veri a, Veri b)
    {
        return new Veri(a.Deger + b.Deger);
    }
}
```

Yaygin Hata

Operator overload metodu `static` olmak zorundadir. Bu kural unutulursa derleme hatasi alinir.

9.4.4 is / as ile Guvenli Downcast

```
C#
HayvanBase hb = new Kedi();

if (hb is Kedi k)
{
    k.Miyavla();           // pattern matching
}

Kopek kp = hb as Kopek;    // basarisiz olursa null
if (kp != null)
{
    kp.Havla();
}
```

Akilda Kalsin

Kisa fark: `is` true/false kontrol eder (ve pattern ile degisken verir), `as` ise cast basarisizsa exception yerine `null` dondurur.

10 Tasarim Desenleri (Design Patterns)

> Tasarim desenleri, tekrarlanan tasarim problemlerine denenmis ve test edilmis cozumler sunan **yontemler topluluğudur**. Bir kurallar topluluğu degildir; gercek tasarim sanatlari.

10.1 GoF (Gang of Four) Sinflandirmasi

GoF Tasarim Desenleri (toplam 23)

Kategori	Amac	Ornekler
Yaratim (Creational)	Nesnelerin nasil olusturulacagini soyutlar.	Singleton, Abstract Factory, Factory Method, Builder, Prototype
Yapisal (Structural)	Sinif/nesneleri bir araya getirir, daha buyuk yapilar olusturur.	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Davranissal (Behavioral)	Sinif/nesneler arasi etkilesimi tanimlar.	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

10.2 Singleton - Detayli

Singleton

Bir siniftan **yalnizca tek bir nesne** uretilmesini garanti eden, ve bu tek nesneye uygulamanin her yerinden erisilebilen tasarim desenidir.

```C#
public sealed class Singleton
{
    // 1) Kendi sinifimizdan static private bir referans
    private static Singleton instance;

    // 2) Lock objesi (multi-thread guvenligi)
    private static readonly object kilit = new object();

    // 3) Kurucu PRIVATE (disaridan new ile uretim engellenir)
    private Singleton() { }

    // 4) Erisim noktasi
    public static Singleton Olustur()
    {
        if (instance == null)
        {
            lock (kilit)
            {
                if (instance == null)              // double-checked locking
                    instance = new Singleton();
            }
        }
        return instance;
    }

    public void Yaz() { Console.WriteLine("Tek nesne"); }
}

// Kullanim
var a = Singleton.Olustur();
var b = Singleton.Olustur();
// a ve b ayni nesnedir!
```

Singleton dogru kullanildiginda faydalidir ama asiri kullanilirsa global state sorunlari uretir. Ozellikle test edilebilirlik ve bagimlilik yonetimi zorlasabilir. Sinavda avantajini yazarken tek ornek gereksinimini, riskini yazarken global baglilik olusumunu belirtmek dengeli cevap olur.

> **Akilda Kalsin**
>
> Singleton ne zaman kullanilir? Loglama, konfigurasyon yoneticisi, database connection pool, cache, Application context, OS uzerinde 1 tane olmasi gereken servisler.

10.3 Diger Onemli Desenler - Kisa Aciklama

- **Factory Method**: Hangi alt sinifin olusturulacagina alt siniflar karar verir. `Vehicle Create(string type)`.
- **Observer**: Bir nesnedeki degisiklik baska nesnelere bildirilir (event sistemi temeli).
- **Strategy**: Algoritmayi nesne olarak sarmalar, calisma aninda secilir. Interface + farkli concrete siniflar.
- **Adapter**: Iki uyumsuz interface'i birbirine baglar.
- **Decorator**: Nesneye dinamik olarak yeni ozellik ekler (kalitim olmadan).

- **Iterator**: Koleksiyon icinden gezinme; `IEnumerable` ve `foreach`'in temeli.

> **Sinavda Cikarsa**
>
> Tasarim desenleri sinavda genelde MCQ veya 'Singleton'u kodla ifade ediniz' acik uclu olarak sorulur. Singleton ezbere yazilabilmelidir.

Ozet sonu - Iyi sinavlar!